GalaxyBridge CTF

22PC03-Abinaya B
22PC29-Samrithi V

Challenge 1 — Sputnik

Overview

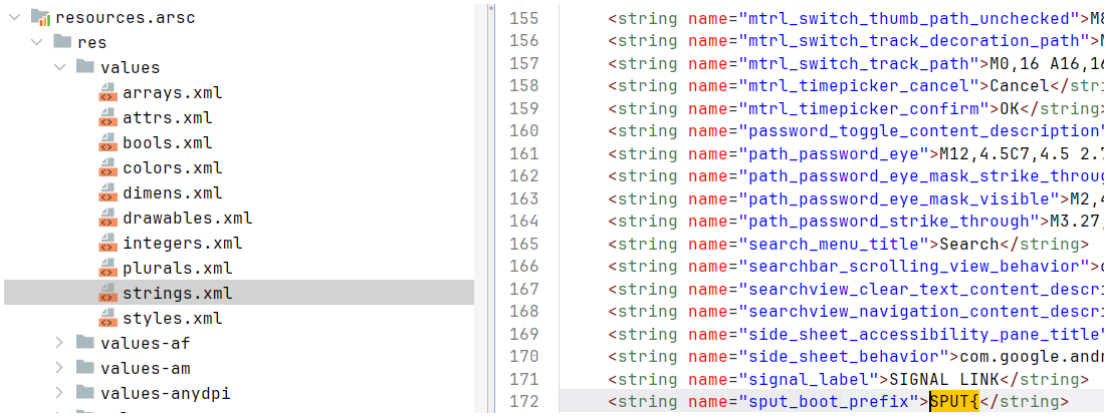
The flag is split into three parts and stored across three different files inside the APK. There is no encoding or decoding involved. You just need to open the APK in JADX, find each part, and join them together.

Step 1 — Open the APK in JADX

Open JADX GUI and load the APK file. Once it loads, go to the Resources tab on the left panel. This is where you will find all the raw resource files.

Step 2 — Get Part 1 from strings.xml

In JADX, navigate to Resources > res > values > strings.xml. Look for a string entry related to sputnik. The value of that entry is the first part of the flag.



Step 3 — Get Part 2 from arrays.xml

In JADX, navigate to Resources > res > values > arrays.xml. Find the array entry related to the flag. One of the items in that array is the second part of the flag.

```
arrays.xml
1 <?xml version="1.0" encoding="utf-8"?>
2 <resources>
3     <array name="sputnik_telemetry_segments">
4         <item>L1NK_</item>
5         <item>4CQU</item>
6         <item>1R</item>
7     </array>
8 </resources>
```

Step 4 — Get Part 3 from sputnik.txt

In JADX, navigate to Resources > res > raw > sputnik_boot_patch.txt. Open that file. The text inside it is the third and final part of the flag.

```
> mipmap-xxhdpi
> mipmap-xxxhdpi
v raw
    sputnik_boot_patch.txt
    supernova
```

```
sputnik_boot_patch.txt
1 3D}
```

Step 5 — Combine and Submit

Put the three parts together in the correct order to form the complete flag. Enter it in the Node 1 terminal in the app. No commands or scripts are needed for this challenge.

Flag 1: SPUT{L1NK_4CQU1R3D}

Challenge 2 — Moon Base Alpha

Step 1 — Open the APK in JADX

Open the APK in JADX and go to the Resources tab. Navigate to assets and locate the file called moon_base_archive.bin. Read the Node2MoonBase class in the source to understand how the file is structured — it shows the header format, the key byte position, and the XOR decode logic.

Step 2 — Extract the APK

Ubuntu:

```
unzip -q app-debug.apk -d tmp_apk
```

Windows:

```
Expand-Archive -LiteralPath app-debug.apk -DestinationPath tmp_apk
```

Step 3 — Locate the Asset

Ubuntu:

```
ls tmp_apk/assets/
```

Windows:

```
dir tmp_apk\assets\
```

You will see moon_base_archive.bin. The file is 24 bytes. The first 4 bytes are the header MBAF, the 5th byte is the XOR key, and the remaining bytes are the encoded flag.

```
PS C:\Users\samrithi> $apk = "F:\Download\Mobile Security Package -2\GalaxyBridge\app\build\outputs\apk\debug\app-debug.apk"
PS C:\Users\samrithi> $zip = "F:\Download\Mobile Security Package -2\GalaxyBridge\tmp_app.zip"
PS C:\Users\samrithi> $dest = "F:\Download\Mobile Security Package -2\GalaxyBridge\tmp_apk"
PS C:\Users\samrithi> Copy-Item $apk $zip
PS C:\Users\samrithi> Remove-Item -Recurse -Force $dest -ErrorAction SilentlyContinue
PS C:\Users\samrithi> Expand-Archive -LiteralPath $zip -DestinationPath $dest
PS C:\Users\samrithi>  Remove-Item $zip
```

Step 4 — Verify the File

Ubuntu:

```
xxd tmp_apk/assets/moon_base_archive.bin
```

Windows (PowerShell):

```
Format-Hex tmp_apk\assets\moon_base_archive.bin
```

Confirm the first 4 bytes read 4D 42 41 46 which is MBAF in ASCII and the 5th byte is 5A which is the XOR key.

Step 5 — Decode the Flag

Create a file called decode_node2.py and run it.

```
from pathlib import Path

data = Path("tmp_apk/assets/moon_base_archive.bin").read_bytes()
key = data[4]
payload = data[5:]
flag = bytes(b ^ key for b in payload).decode("utf-8")
print(flag)
```

Ubuntu: `python3 decode_node2.py` Windows: `python decode_node2.py`

```
PS C:\Users\samrithi> python .\script_Node6.py
MOON{B4SE_C0MM_L0G}
```

Step 6 — Submit

Enter the decoded string in the Node 2 terminal in the app.

Flag 2 : MOON{B4SE_C0MM_L0G}

Challenge 3 — Mars

Overview

The app has an exported broadcast receiver called MarsRoverReceiver. You trigger it by sending a broadcast via ADB. The debug build has a different package name than the release build, so you must confirm the installed name first.

Step 1 — Confirm the Installed Package Name

Ubuntu:

```
adb shell pm list packages | grep galaxybridge
```

Windows:

```
adb shell pm list packages | findstr galaxybridge
```

If you installed the debug APK, the result will be **com.galaxybridge.debug**. This is because the debug build type adds applicationIdSuffix = ".debug" in build.gradle.

Step 2 — Start Logcat First

Open a terminal and run this before sending the broadcast:

Ubuntu and Windows:

```
adb logcat -s ROVER_SIGNAL
```

Step 3 — Send the Broadcast

In a second terminal window, run:

```
adb shell am broadcast -a com.galaxybridge.ACTION_PING_ROVER -n
com.galaxybridge.debug/com.galaxybridge.receivers.MarsRoverReceiver
```

```
PS C:\Users\samrithi> adb shell am broadcast -a com.galaxybridge.ACTION_PING
_ROVER -n com.galaxybridge.debug/com.galaxybridge.receivers.MarsRoverReceive
r
Broadcasting: Intent { act=com.galaxybridge.ACTION_PING_ROVER flg=0x400000 c
mp=com.galaxybridge.debug/com.galaxybridge.receivers.MarsRoverReceiver }
Broadcast completed: result=0
```

Expected Output

```
D/ROVER_SIGNAL: PING received — FLAG{...}
```

```
PS C:\Users\samrithi> adb logcat -s ROVER_SIGNAL
--------- beginning of main
04-06 11:17:27.887  3719  3719 D ROVER_SIGNAL: PING RECEIVED — initiating handshake sequence...
04-06 11:17:27.890  3719  3719 D ROVER_SIGNAL: Authenticating remote source...
04-06 11:17:27.890  3719  3719 D ROVER_SIGNAL: Triangulating position... LAT: -4.5895°  LON: 137.4417°
04-06 11:17:27.890  3719  3719 D ROVER_SIGNAL: Atmospheric pressure: 0.00628 atm
04-06 11:17:27.890  3719  3719 D ROVER_SIGNAL: Surface temperature: -60°C
04-06 11:17:27.891  3719  3719 D ROVER_SIGNAL: Power systems: NOMINAL
04-06 11:17:27.891  3719  3719 D ROVER_SIGNAL: AUTH OVERRIDE ACCEPTED — TRANSMITTING SECURE CODE:
04-06 11:17:27.891  3719  3719 D ROVER_SIGNAL: MARS{R0V3R_R3SP0NDS}
04-06 11:17:27.892  3719  3719 D ROVER_SIGNAL: LINK RE-ESTABLISHED. ROVER SYSTEMS ONLINE.
04-06 11:17:27.893  3719  3719 D ROVER_SIGNAL: Uploading terrain scan: 2381 KB
04-06 11:17:27.893  3719  3719 D ROVER_SIGNAL: Camera mast: 73.2° azimuth — survey complete
```

Rooting the device is not required. This is a normal exported receiver.

Flag 3: MARS{R0V3R_R3SP0NDS}

Challenge 4 — Saturn

Overview

The app stores a JPEG image in external storage. The flag is hidden inside the image's EXIF metadata, not in the image itself.

Step 1 — List the Directory on the Device

Ubuntu and Windows:

```
adb shell ls
/sdcard/Android/data/com.galaxybridge.debug/files/probe_imaging
```

Expected result: cassini_imaging.jpg

Step 2 — Pull the Image

Ubuntu and Windows:

```
adb pull
/sdcard/Android/data/com.galaxybridge.debug/files/probe_imaging/cassini_ima
ging.jpg .
```

```
PS C:\Users\samrithi> adb shell ls /sdcard/Android/data/com.galaxybridge.debug/files/probe_imaging
cassini_imaging.jpg
PS C:\Users\samrithi> adb pull /sdcard/Android/data/com.galaxybridge.debug/files/probe_imaging/cassini_imaging.jpg
/sdcard/Android/data/com.galaxybridg...ed. 1.5 MB/s (23194 bytes in 0.015s)
```

Step 3 — Read the EXIF Metadata

Option A — Online tool:

Go to https://www.metadata2go.com/view-metadata and upload the image. Look through all metadata fields for the flag.

Option B — Command line (Ubuntu):

```
sudo apt install exiftool -y
exiftool cassini_imaging.jpg
```

The flag will appear in one of the metadata fields such as Comment, UserComment, or ImageDescription.

checksum	8ca47115ad5da14553ff1e7a3f62106a
file_name	cassini_imaging.jpg
file_size	23 kB
file_type	JPEG
file_type_extension	jpg
mime_type	image/jpeg
exif_byte_order	Big-endian (Motorola, MM)
model	LENS{0B1M4G3_M3T4}
make	GALILEO_IMAGING_SCIENCE
software	CASSINI-PROBE-OS/3.2.1
image_description	Saturn Ring Survey ? Mission Sol 4827
light_source	Unknown
orientation	Unknown (0)
modify_date	2004:01:01 00:00:00

Step 4 — Submit

Enter the flag string in the Node 4 terminal in the app.

Flag 4: LENS{0B1M4G3_M3T4}

Challenge 5 — Voyager Mission

Overview

You need to launch a headless activity via ADB and pass it two specific string extras. The activity runs, logs the flag, and closes immediately. You capture the flag from logcat.

Step 1 — Start Logcat First

Open a terminal and run this before launching the activity:

Ubuntu and Windows:

```
adb logcat -s VOYAGER_MISSION
```

Step 2 — Launch the Activity

In a second terminal, run:

Ubuntu:

```
adb shell am start -a com.galaxybridge.ACTION_VOYAGER_MISSION -n
com.galaxybridge.debug/com.galaxybridge.challenges.Node5VoyagerActivity
--es probe_id VOYAGER --es mission_code GOLDEN_RECORD
```

Windows:

```
adb shell am start -a com.galaxybridge.ACTION_VOYAGER_MISSION -n
com.galaxybridge.debug/com.galaxybridge.challenges.Node5VoyagerActivity
--es probe_id VOYAGER --es mission_code GOLDEN_RECORD
```

```
PS C:\Users\samrithi> adb shell am start -a com.galaxybridge.ACTION_VOYAGER_MISSION -n com.galaxybridge.debug/com.galaxy
bridge.challenges.Node5VoyagerActivity --es probe_id VOYAGER --es mission_code GOLDEN_RECORD
Starting: Intent { act=com.galaxybridge.ACTION_VOYAGER_MISSION cmp=com.galaxybridge.debug/com.galaxybridge.challenges.No
de5VoyagerActivity (has extras) }
```

Expected Logcat Output

```
I/VOYAGER_MISSION: MISSION REPLY: FLAG{...}
```

The activity has no UI. Nothing will appear on the device screen. The flag only appears in logcat.

```
PS C:\Users\samrithi> adb logcat -s VOYAGER_MISSION
--------- beginning of main
04-06 11:35:53.811  9697  9697 I VOYAGER_MISSION: MISSION REPLY: VOY{INT3NT_R3LAY}
```

Step 3 — Submit

Enter the flag string in the Node 5 terminal in the app.

Flag 5 : VOY{INT3NT_R3LAY}

Challenge 6 — Nebula

Overview

The flag is stored inside the APK's assets folder as an encoded text file. The encoding uses Base64 followed by XOR. No need to run the app — everything is done statically using JADX.

Step 1 — Read the Source in JADX

Open the APK in JADX. In the Source Code tab, navigate to com.galaxybridge.challenges.Node6Nebula and read the getFlag() method. It tells you the file is at assets/random/nebula.txt and the decode steps are Base64 then XOR.

Step 2 — Get the Encoded String from JADX

In JADX, go to the Resources tab and navigate to assets > random > nebula.txt. Open the file and copy the Base64 string inside it.

Step 3 — Decode the Flag

Ubuntu and Windows — run this Python script (script_Node6.py):

```
import base64

# The encrypted data from assets/random/nebula.txt
encoded = 'AQoNGjQMfxwCfgwQA38MBDI='

# Base64 decode
decoded_bytes = base64.b64decode(encoded)

# XOR with key 0x4F
flag = ''.join(chr(b ^ 0x4F) for b in decoded_bytes)

print(flag)
```

Run it:

```
python3 decode_nebula.py
```

On Windows use: python script_node6.py

```
C:\Users\samrithi>python script_Node6.py
NEBU{C0SM1C_L0CK}
```

Step 4 — Submit

Enter the decoded flag in the Node 6 terminal in the app.

Flag 6 : NEBU{C0SM1C_L0CK}

Challenge 7 — Supernova

Overview

The flag is logged to logcat when you tap Node 7 in the app. The log tag is COSMIC_BLAST.

Step 1 — Start logcat with the right filter:

Ubuntu and Windows:

```
adb logcat -s COSMIC_BLAST
```

Or use:

```
adb logcat | grep COSMIC_BLAST
```

Step 2 — Open the app and tap Node 7. The flag will appear in the log:

```
D/COSMIC_BLAST: Cosmic flag: <flag>
```

The logged value is already the decoded flag. Copy it and submit.

```
C:\Users\samrithi>adb logcat -s COSMIC_BLAST
--------- beginning of main
04-06 12:19:46.613 11134 11134 D COSMIC_BLAST: Cosmic flag: SUPERNOVA{EMBER_TRACE}
```

Step 3 — Submit

Enter the flag in the Node 7 terminal in the app.

Flag 7 : SUPERNOVA{EMBER_TRACE}

Challenge 8 — Black Hole

Overview

The flag is AES-encrypted and stored in SharedPreferences when you tap Node 8 in the app. You need to extract the encrypted value at runtime using ADB, then decrypt it using the hardcoded key found in the APK.

Step 1 — Decompile the APK to find the key

Use JADX or APKTool to decompile the APK and locate `Node8BlackHole.java`. You'll find:

- **Encryption Key:** <key>
- **Algorithm:** `<Algorithm>`
- **Storage file:** `<Locate the xml file>`

Step 2 — Trigger the flag storage

Open the app and **tap Node 8**. This runs `storeEncryptedFlag()` internally, which encrypts the flag and writes it to SharedPreferences.

Step 3 — Extract the encrypted value via ADB

Run the following command:

```
adb shell run-as com.galaxybridge.debug cat shared_prefs/blackhole_prefs.xml
```

Look for the value inside:

```Xml
<string
name="encrypted_flag">6j2esd4Fus57lHmsRGss3BWMrJVXWOdX6fUtAdHSMsI=</strin
g>
```

Copy that Base64 string.

Step 4 — Decrypt the flag

Run this Python script with the extracted Base64 value:

Python

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
import base64

encrypted_b64 = "6j2esd4Fus57lHmsRGss3BWMrJVXWOdX6fUtAdHSMsI="  # paste your extracted value here
key = b"BlackHoleKey1234"

cipher = Cipher(algorithms.AES(key), modes.ECB(), backend=default_backend())
decryptor = cipher.decryptor()
encrypted = base64.b64decode(encrypted_b64)
decrypted = decryptor.update(encrypted) + decryptor.finalize()

flag = decrypted.decode('utf-8').rstrip('\x00')
print(flag)
```

```
C:\Users\samrithi>adb shell run-as com.galaxybridge.debug cat shared_prefs/blackhole_prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="encrypted_flag">6j2esd4Fus57lHmsRGss3BWMrJVXWOdX6fUtAdHSMsI=&#10;    </string>
</map>

C:\Users\samrithi>python script_Node6.py
BLACKHOLE{EVENT_HORIZON}
```

Step 5 — Submit

Enter the flag in the Node 8 terminal in the app.

Flag 8 : BLACKHOLE{EVENT_HORIZON}